

Lecture 6: Long-Short Term Memory (LSTM) Neural Networks

Yaxin Bi
School of Computing
University of Ulster

1

Contents

- ▶ Issues with Recurrent Neural Networks
- ▶ Architectures of Long-Short Term Memory (LSTM) Neural Networks
- ▶ Internal States of LSTMs
- ▶ Variants of LSTMs
- ▶ Sequence Prediction Problems

2

Problems of Recurrent Neural Networks

- ▶ Recurrent Neural Networks (RNNs) suffer from short-term memory
 - ▶ If a sequence is long enough, RNNs will have a difficulty carrying information from earlier time steps to later ones.
 - ▶ If use an RNN to process a paragraph of text and do predictions, it may leave out important information from the beginning.
- ▶ During back propagation, RNNs suffer from the vanishing gradient problem.
 - ▶ Gradient values are used to update RNN weights.
 - ▶ The vanishing gradient problem occurs when the gradient shrinks as it back propagates through time.
 - ▶ If a gradient value becomes extremely small, it doesn't contribute too much learning.

3

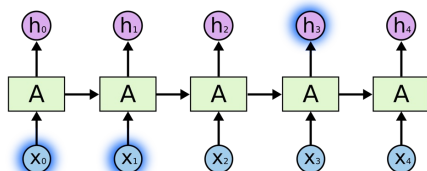
The Problem of Long-Term Dependencies

- ▶ One of the appeals of RNNs is that they are able to connect previous state to the present input, such as using previous character 'h' might inform the understanding of the present character 'e' at the Character-level Language Model.
- ▶ Sometimes, we only need to look at recent information to perform the present task. For example, consider a language model trying to predict the next word based on the previous ones.
- ▶ If we are trying to predict the last word in "the clouds are in the sky," we don't need any further context – it's pretty obvious the next word is going to be 'sky'.

4

The Problem of Long-Term Dependencies (cont'd)

- ▶ In such cases, where the gap between the relevant information and the place that it's needed is small, RNNs can learn to use the past information.



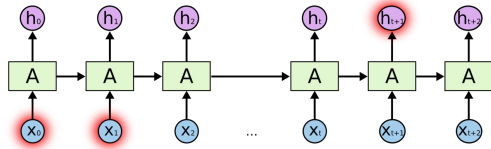
5

The Problem of Long-Term Dependencies (cont'd)

- ▶ But there are also cases where we need more context.
 - ▶ Consider predicting the last word in the text "**I grew up in France... I speak fluent French.**"
- ▶ Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back.
- ▶ It's entirely possible for the gap between the relevant information and the point where it is needed to become very large.
- ▶ Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.

6

The Problem of Long-Term Dependencies (cont'd)



- ▶ In theory, RNNs are absolutely capable of handling such "long-term dependencies." A human could carefully pick parameters for them to solve toy problems of this form. Unfortunately, in practice, RNNs don't seem to be able to learn them.

7

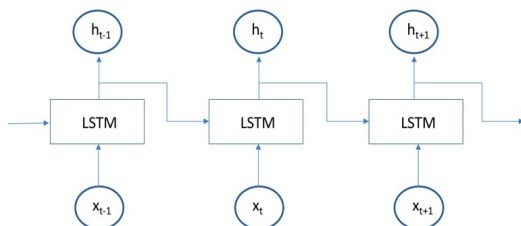
Long-Short Term Memory (LSTM) networks

- ▶ Long-Short Term Memory networks – usually called LSTMs – are a special kind of RNNs, capable of learning long-term dependencies.
- ▶ They work tremendously well on a large variety of problems, which are now widely used.
- ▶ LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn!
- ▶ All RNNs have the form of a chain of repeating modules (neurons) of neural network.
- ▶ In standard RNNs, this repeating module normally has a very simple structure, such as a single tanh layer.

8

A simple structure of LSTM

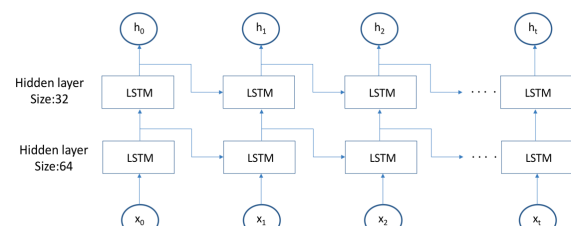
- ▶ A simple structure of LSTM with return_sequences=True



9

Two Hidden Layers

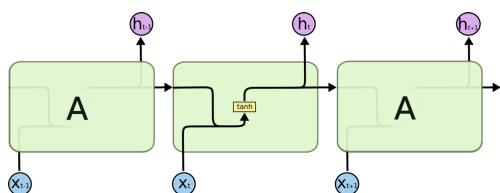
- ▶ A simple structure of LSTM with return_sequences=True with two layers



10

Standard RNNs

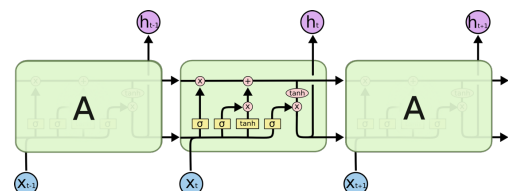
- ▶ In standard RNNs, this repeating module (neuron) will have a very simple structure, such as a single tanh layer.



11

Detailed LSTM Structure

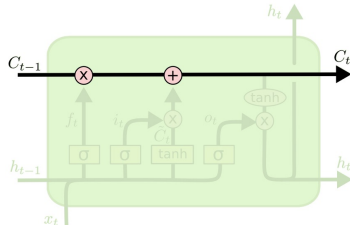
- ▶ LSTMs also have this chain like structure, but the repeating module has a different structure compared with RNNs. Instead of having a single neural network layer, there are four, interacting in a very special way.



12

The Key Idea Behind LSTMs

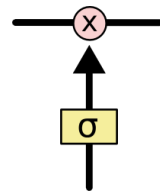
- ▶ The key to LSTMs is the cell state, the horizontal line running through the top of the diagram.
- ▶ The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.



13

The Cell State of a LSTM

- ▶ The LSTMs have the ability to remove or add information to the cell state, carefully regulated by structures called gates.
- ▶ Gates are a way of optionally letting information through. They are composed of a sigmoid neural net layer and a pointwise multiplication operation.
- ▶ The sigmoid layer outputs numbers between 0 and 1, representing how much of each component should be let through.
- ▶ 0 means "let nothing through," while 1 means "let everything through!"



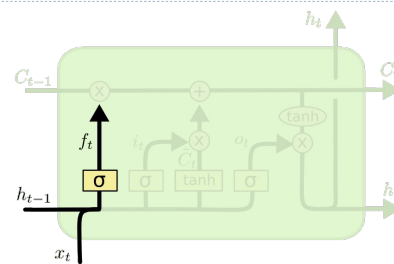
14

Forget Gates in LSTMs

- ▶ The first step in LSTMs is to decide what information will be thrown away from the cell state.
- ▶ This decision is made by a sigmoid layer called the "forget gate layer."
- ▶ It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} .
- ▶ A 1 represents "completely keep this" while a 0 represents "completely get rid of this."
- ▶ Let's see the example of a language model, predicting the next word based on all the previous ones.
- ▶ In such a problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used.
- ▶ When seeing a new subject, we want to forget the gender of the old subject.

15

Forget Gate (cont'd)



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

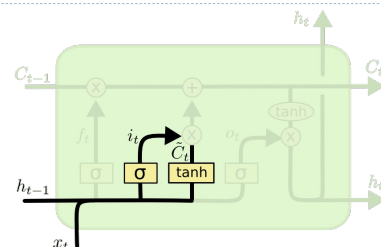
16

Input Gate Layer in LSTMs

- ▶ The next step is to decide what new information the module is going to store in the cell state. This has two parts.
 - ▶ First, a sigmoid layer called the "input gate layer" decides which values will be updated.
 - ▶ Second, a tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.
- ▶ In the subsequent step, the module will combine these two to create an update to the state.
- ▶ In the example of the language model, the module adds the new subject to the cell state, to replace the old subject of the gender that the module is forgetting.

17

Input Gate Layer in LSTMs (cont'd)



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

18

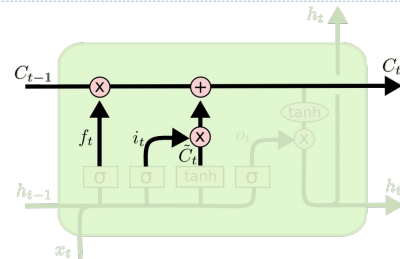
Cell State and Aggregation

- ▶ Update the old cell state, C_{t-1} , into the new cell state C_t .
- ▶ The previous steps already decided what to do, i.e. multiply the old state by f_t , forgetting the things the module decides to forget earlier.
- ▶ Then add $i_t * \tilde{C}_t$. This is the new candidate values, scaled by how much the module decides to update each state value.
- ▶ In the case of the language model, this is where the module drops the information about the old subject's gender and adds the new information, as decided in the previous steps.

19

19

Cell State and Aggregation



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

20

20

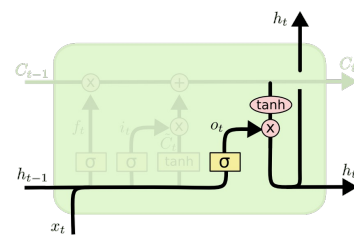
LSTM Output and Output Gate

- ▶ The final step is to decide what the module is going to output. This output will be based on the cell state, but will be a filtered version.
- ▶ First run a sigmoid layer to decide what parts of the cell state the module is going to output.
- ▶ Then put the cell state through tanh (values are between -1 and 1) and multiply it by the output of the sigmoid gate, hence only output the parts the module decides to.
- ▶ For the language model example, since it saw a subject, it might want to output information relevant to a verb, in case that's what is coming next.
 - ▶ for instance, output whether the subject is singular or plural, so that predicting what form a verb should be conjugated into if that's what follows next.

21

21

LSTM Output and Output Gate (cont'd)



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

22

22

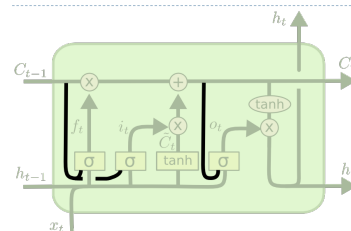
Variant 1 on the LSTM

- ▶ In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.
- ▶ What I've introduced so far is a normal LSTM. But not all LSTMs are the same as the above. It seems like almost every paper involving LSTMs uses a slightly different version. The differences are minor, but it's worth mentioning some of them.
- ▶ One popular LSTM variant, introduced by [Gers & Schmidhuber \(2000\)](#), is adding "peephole connections", letting the gate layers look at the cell state.
- ▶ The above diagram adds peepholes to all the gates, but many papers will give some peepholes and not others.

23

23

Variant 1 on the LSTM



$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

24

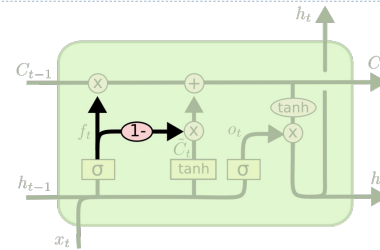
24

Variant 2 on the LSTM

- ▶ Another variation is to use coupled forget and input gates.
- ▶ Instead of separately deciding what to forget and what the module should add new information to, this variant makes those decisions together.
- ▶ Only forget when the module is going to input something in its place.
- ▶ Only input new values to the state when forgetting something older.

25

Variant 2 on the LSTM



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

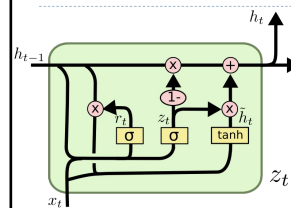
26

Variant 3 on the LSTM: GRU

- ▶ A slightly more dramatic variation on the LSTM is the Gated Recurrent Unit, or GRU, introduced by [Cho, et al. \(2014\)](#).
- ▶ It combines the forget and input gates into a single “update gate.”
- ▶ It also merges the cell state and hidden state, and makes some other changes.
- ▶ The resulting model is simpler than standard LSTM models, and has been growing increasingly popular.

27

Variant 3 on the LSTM: GRU



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

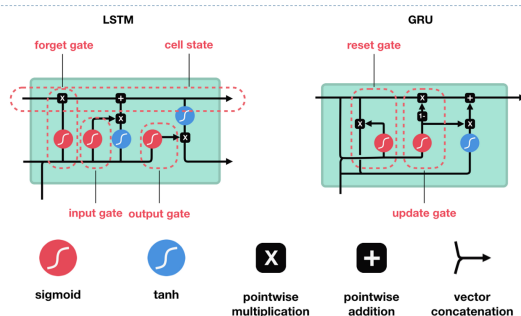
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

28

LSTM and GRU



29

Keras API for LSTMs

- ▶ In the Keras deep learning library, LSTM layers can be created using the [LSTM\(\) class](#).
- ▶ Creating a layer of LSTM memory units allows you to specify the number of memory units within the layer.
- ▶ Each unit or cell within the layer has an internal cell state, often abbreviated as “c”, and outputs a hidden state, often abbreviated as “h”.
- ▶ The Keras API allows to access these data, which can be useful or even required when developing sophisticated RNN architectures, such as the encoder-decoder model.

30

Keras API Access to Internal State

- ▶ Setting “return sequences” in the layer returns the hidden state output for each input time step.
- ▶ Setting “return state” returns the hidden state output and cell state for the last input time step.
- ▶ Setting “return sequences and return state” can be used at the same time and return both

31

Example: Return Sequence

- ▶ Given one input sample with 3 time steps and one feature observed at each time step, return sequence, $t_1 = 0.1$, $t_2 = 0.2$, $t_3 = 0.3$

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input
from tensorflow.keras.layers import LSTM
from numpy import array
# define model
inputs1 = Input(shape=(3, 1))
lstm1 = LSTM(1)(inputs1)
model = Model(inputs=inputs1, outputs=lstm1)
# define input data
data = array([0.1, 0.2, 0.3]).reshape((1,3,1))
# make and show prediction
print(model.predict(data))
```

```
[[[-0.0953151]]]
```

32

Example: Return Sequence (cont'd)

- ▶ Access the hidden state output for each input time step by setting the return_sequences attribute to True when defining the LSTM layer, as follows:

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input
from tensorflow.keras.layers import LSTM
from numpy import array
# define model
inputs1 = Input(shape=(3, 1))
lstm1 = LSTM(1, return_sequences = true)(inputs1)
model = Model(inputs=inputs1, outputs=lstm1)
# define input data
data = array([0.1, 0.2, 0.3]).reshape((1,3,1))
# make and show prediction
print(model.predict(data))
```

```
[[[-0.02243521] [-0.06210149] [-0.11457888]]]
```

33

Example: Return State

- ▶ Given one input sample with 3 time steps and one feature observed at each time step, return state, $t_1 = 0.1$, $t_2 = 0.2$, $t_3 = 0.3$

```
from keras.models import Model
from keras.layers import Input
from keras.layers import LSTM
from numpy import array
# define model
inputs1 = Input(shape=(3, 1))
lstm1, state_h, state_c = LSTM(1, return_state=True)(inputs1)
model = Model(inputs=inputs1, outputs=[lstm1, state_h, state_c])
# define input data
data = array([0.1, 0.2, 0.3]).reshape((1,3,1))
# make and show prediction
print(model.predict(data))
```

34

Example: Return State (cont'd)

- ▶ Running the example returns 3 arrays:
 - ▶ The LSTM hidden state output for the last time step.
 - ▶ The LSTM hidden state output for the last time step (again).
 - ▶ The LSTM cell state for the last time step.

```
[array([[ 0.10951342]], dtype=float32),
 array([[ 0.10951342]], dtype=float32),
 array([[ 0.24143776]], dtype=float32)]
```

35

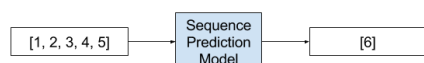
Sequence Prediction

- ▶ Sequence prediction is different to other types of supervised learning problems.
- ▶ The sequence imposes an order on the observations that must be preserved when training models and making predictions.
- ▶ There are 4 main types of sequence prediction problems:
 - ▶ Sequence Prediction
 - ▶ Sequence Classification
 - ▶ Sequence Generation
 - ▶ Sequence-to-Sequence Prediction

36

Sequence Prediction

- ▶ Given an input sequence, predict the next value in the sequence.



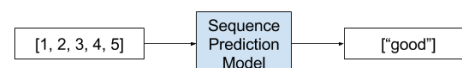
- ▶ Applications include
 - ▶ Weather forecasting
 - ▶ Sales prediction
 - ▶ Product recommendation
 - ▶ Energy consumption

37

37

Sequence Classification

- ▶ Given an input sequence, classify the sequence.



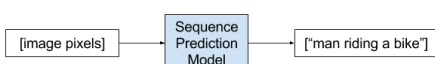
- ▶ Possible applications include
 - ▶ DNA sequence classification
 - ▶ Anomaly detection
 - ▶ Sentiment analysis

38

38

Sequence Generation

- ▶ Given an observation, generate an output sequence.



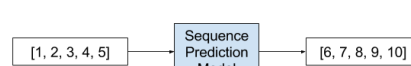
- ▶ Applications include
 - ▶ Text generation
 - ▶ Music generation
 - ▶ Image captioning

39

39

Sequence to Sequence Prediction

- ▶ Given an input sequence, generate an output sequence.



- ▶ Applications include
 - ▶ Multi-Step time series forecasting
 - ▶ Text summarization
 - ▶ Language translation

40

40

Summary

- ▶ The previous lecture introduces RNNs, the remarkable results people are achieving with LSTMs. They really work a lot better for most tasks
- ▶ LSTMs were developed with a set of mathematic equations, which were a big step in what we can accomplish with RNNs.
- ▶ The idea is to let every step of an RNN pick information to look at from some larger collection of information. For example, if you are using an RNN to create a caption describing an image, it might pick a part of the image to look at for every word it outputs. In fact, it might be a good starting point if you want to explore attention!
- ▶ These variants introduced here are only a few of the most notable LSTM ones. There are lots of others. There's also some completely different approaches to tackling long-term dependencies
- ▶ Which of these variants is best? Do the differences matter?
- ▶ More than ten thousand RNN architectures were developed, finding some that worked better than LSTMs on certain tasks.

41

41